

1 论文精读: E-GRPO

论文: [E-GRPO: High Entropy Steps Drive Effective Reinforcement Learning for Flow Models](#)

作者: Shengjun Zhang, Zhang Zhang, Chensheng Dai, Yueqi Duan。

简称: E-GRPO。

版本: arXiv 2601.00423v1。

代码: [VisualGRPO](#)。

方向标签: Flow Matching、GRPO、Entropy-aware Exploration、Step Merging、Visual Preference Alignment。

阅读日期: 2026-05-06。

阅读口径: 这篇和 OP-GRPO 都属于 “sample efficiency / exploration efficiency” 支线, 但切入点不同: OP-GRPO 问 **旧轨迹能不能复用**, E-GRPO 问 **哪些 denoising steps 真正值得随机探索和做 RL 更新**。

1.1 0. 一句话结论

E-GRPO 的核心判断很直接: Flow-GRPO 不应该在所有 denoising steps 上均匀做 SDE 探索和 GRPO 更新, 因为不同时间步的探索熵不一样。

E-GRPO = entropy-aware step selection
+ low-entropy step merging
+ multi-step group-normalized advantage

它认为 high-entropy steps 会产生更有区分度的 rollouts, 更适合用 reward 做相对比较; low-entropy steps 的随机扰动太小, 生成结果差异不明显, 继续把这些 steps 当成独立 RL action 只会增加计算和 reward attribution 噪声。

我的判断: E-GRPO 不是在改 reward, 也不是在做 replay buffer, 而是在改 **探索发生的位置**。它把 Flow-GRPO 里的 “每一步都随机、每一步都更新” 改成 “只在高信息量的 consolidated SDE step 上随机和更新”。这使它和 OP-GRPO 正好互补: 一个提高轨迹复用效率, 一个提高单次 rollout 内部的步骤效率。

1.2 1. 它为什么重要

Flow-GRPO 把 deterministic flow ODE 改写成 stochastic SDE policy, 从而能计算 transition probability、做 GRPO-style policy update。这个改写解决了“flow model 没有 logprob 怎么做 RL”的问题, 但也引入了一个新的默认假设:

所有 denoising steps 都值得注入随机性, 也都值得接受同样形式的 RL 更新。

E-GRPO 认为这个假设是有问题的。因为在 flow / diffusion 采样里, 不同时间步的随机扰动影响完全不同:

- 早期 high-noise / high-entropy steps 会改变整体构图、语义布局和主体形态, 生成结果之间差异大。
- 后期 low-noise / low-entropy steps 更多是在局部细节附近微调, rollouts 之间区分度小。

如果后期低熵步骤产生的几条样本几乎看不出差异, 但我们还强行对它们做 group-relative reward normalization, 那么 reward 里的随机波动、scorer 噪声、下游步骤误差就会被误当成该 step 的学习信号。

关键问题: Flow-GRPO 的稀疏 reward 不只是不知道“哪一步贡献大”, 还不知道“哪一步本来就没有足够探索空间”。如果某一步的 rollout 差异太小, 组内相对优势本身就很难可靠。

所以 E-GRPO 的问题可以概括成:

在 flow-model RL 中, 怎样根据 step entropy 选择真正有探索价值的时间步, 并避免低熵多步随机性带来的 reward attribution ambiguity?

1.3 2. 熵视角: 为什么不是所有 SDE steps 都一样

论文从 Flow-GRPO 的 SDE 采样形式出发。为了让 flow model 可以探索, 原本的 ODE update 会被改写成带随机性的 SDE transition:

$$x_{t+\Delta t} = x_t + \mu_\theta(x_t, t)\Delta t + \sigma_t\sqrt{\Delta t}\epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

其中论文使用的噪声系数形式是:

$$\sigma_t = a\sqrt{\frac{t}{1-t}}.$$

对于反向 SDE step, 它的 transition distribution 可以看成 Gaussian, 因此熵主要由 covariance 控制。论文给出的 step entropy 可写成:

$$h(t) = \frac{d}{2} \log(2\pi e \sigma_t^2 \Delta t).$$

代入 $\sigma_t = a\sqrt{t/(1-t)}$ 后, 有:

$$h(t) = \frac{d}{2} \log\left(2\pi e \cdot a^2 \cdot \frac{t}{1-t} \cdot \Delta t\right).$$

也就是说, 在忽略常数后:

$$e^{h(t)} \propto \frac{t}{1-t} \Delta t.$$

这给了 E-GRPO 一个很明确的判断标准: 哪个 step 的 SDE entropy 更大, 哪个 step 的随机探索范围就更大, 也更可能产生 reward 上可区分的 rollouts。

直觉版: high-entropy step 像是在问“我要往哪个语义方向走”; low-entropy step 更像是在问“边缘再抹平一点还是锐一点”。前者更适合 RL 探索, 后者如果也用同样的 GRPO 强度去训, 收益小、噪声大。

1.4 3. 方法一: Entropy-driven Step Merging

1.4.1 3.1 用阈值区分 high-entropy 和 low-entropy steps

E-GRPO 首先为每个 denoising timestep 计算 entropy proxy, 然后用阈值 τ 将时间步分成两类:

$$\text{high-entropy steps : } e^{h(t_k)} \geq \tau,$$

$$\text{low-entropy steps : } e^{h(t_k)} < \tau.$$

实际 flow sampling 里通常还会做 time shift:

$$t_k = \frac{s\hat{t}_k}{1 + (s-1)\hat{t}_k}, \quad \hat{t}_k = \frac{k}{T}.$$

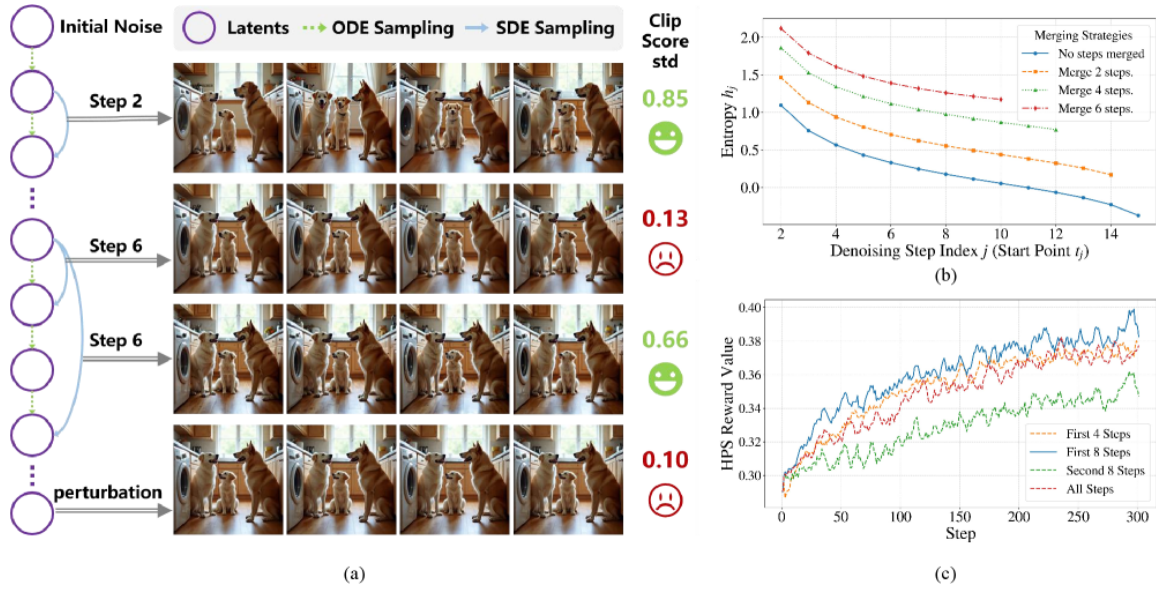


图 1: E-GRPO 对 step entropy 的经验观察。左侧比较不同 SDE sampling 策略下的生成差异, 中间展示合并步数如何提升 entropy, 右侧展示只训练 high-entropy early steps 的 reward 曲线更优。图源: [E-GRPO arXiv HTML Figure 1](#)。

论文将这个时间变换代入 entropy proxy 后, 用它来判断哪些 steps 具有足够探索熵。

1.4.2 3.2 低熵步骤不单独做 SDE, 而是合并成一个更大的 SDE step

对连续低熵步骤 $\{t_m, \dots, t_{m-l}\}$, E-GRPO 不再逐步注入 SDE noise, 而是把它们合并成一个 consolidated SDE step:

$$x_{t_{m-l}} = x_{t_m} + \mu_{\theta}(x_{t_m}, t_m)(t_m - t_{m-l}) + \sigma_t \sqrt{t_m - t_{m-l}} \epsilon.$$

合并后的 entropy 近似满足:

$$e^{h(t_k)} \propto \frac{t_m}{1 - t_m} (t_m - t_{m-l}).$$

因为 $(t_m - t_{m-l})$ 变大, 所以多个低熵小 step 可以合成一个达到阈值附近的高熵 step。

这一步的设计很关键: E-GRPO 不是简单跳过 low-entropy steps, 而是把连续 low-entropy steps 合并成一个“刚好有足够探索空间”的 SDE event。这样既减少了独立随机步骤数量, 又避免完全失去这段轨迹上的探索能力。

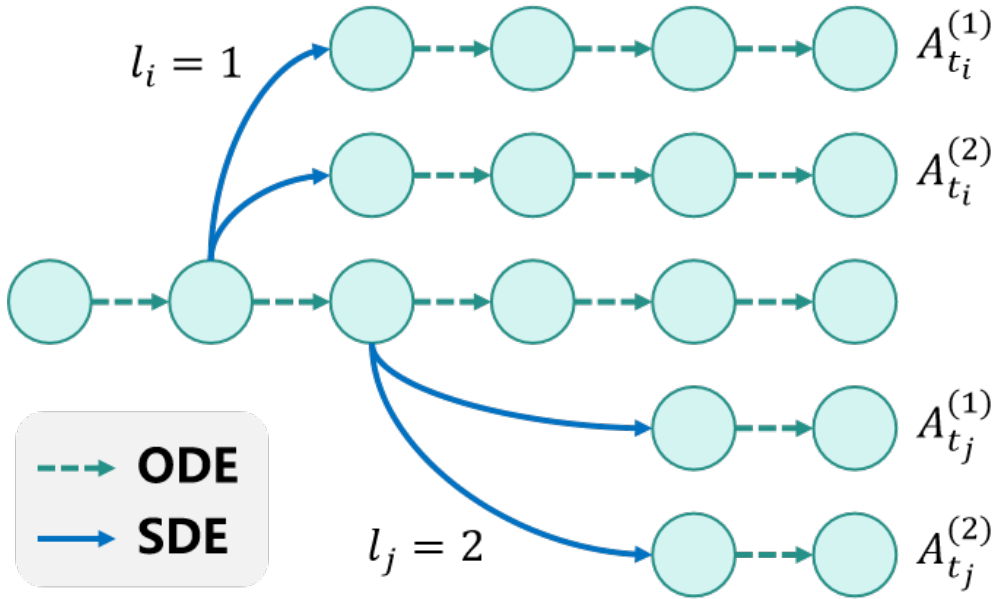


图 2: E-GRPO 的整体 sampling strategy: 先为不同 timesteps 构造 anchor noise latents, 再根据 entropy analysis 为每个 active SDE timestep 选择 merged steps, 最后在对组内生成结果并计算 advantage。图源: [E-GRPO arXiv HTML Figure 2](#)。

1.4.3 3.3 为什么不用固定 2-step / 4-step / 6-step merge

一个简单做法是固定每 l 步合并一次。但 E-GRPO 的消融显示, 固定合并不如 adaptive merging。

原因也很自然: 不同 timestep 的 entropy 本来就不一样。如果固定 4-step merge, 有些位置合并后还是太低熵, 有些位置可能熵过高, 导致探索方向太散。论文因此为每个 low-entropy timestep 自适应选择 l , 让合并后的 entropy 刚好超过阈值 τ 。

$$\text{choose } l \text{ s.t. } e^{h(t_m, l)} \gtrsim \tau.$$

这相当于把每个 consolidated SDE step 的探索尺度校准到一个相近范围。

1.5 4. 方法二: Multi-step Group Normalized Advantage

合并步骤之后, advantage 也不能再按原来的单步方式理解。

对一个 active SDE timestep t_n , 它对应一个合并后的时间段:

$$\mathcal{T}_n = \{t_n, t_{n-1}, \dots, t_{n-l_n}\}.$$

给定同一个 prompt c , 论文为每个 active timestep t_n 生成 $G^{(n)}$ 条 trajectories, 并只在共享同一个 consolidated SDE step 的样本内部计算 group-normalized advantage:

$$A_{t_n}^{(i)} = \frac{R(x_{0,t_n}^{(i)}, c) - \text{mean}\{R(x_{0,t_n}^{(j)}, c)\}_{j=1}^{G^{(n)}}}{\text{std}\{R(x_{0,t_n}^{(j)}, c)\}_{j=1}^{G^{(n)}}}.$$

对应的 ratio 是合并 step 的 transition ratio:

$$r_{t_n}^{(i)}(\theta) = \frac{p_{\theta}(x_{t_n-l_n}^{(i)} | x_{t_n}^{(i)}, c)}{p_{\theta_{\text{old}}}(x_{t_n-l_n}^{(i)} | x_{t_n}^{(i)}, c)}.$$

于是 clipped surrogate 仍然保持 GRPO / PPO 风格:

$$\frac{1}{\hat{T}} \sum_{n=N}^T \frac{1}{G^{(n)}} \sum_{i=1}^{G^{(n)}} \min \left(r_{t_n}^{(i)} A_{t_n}^{(i)}, \text{clip}(r_{t_n}^{(i)}, 1 - \epsilon, 1 + \epsilon) A_{t_n}^{(i)} \right).$$

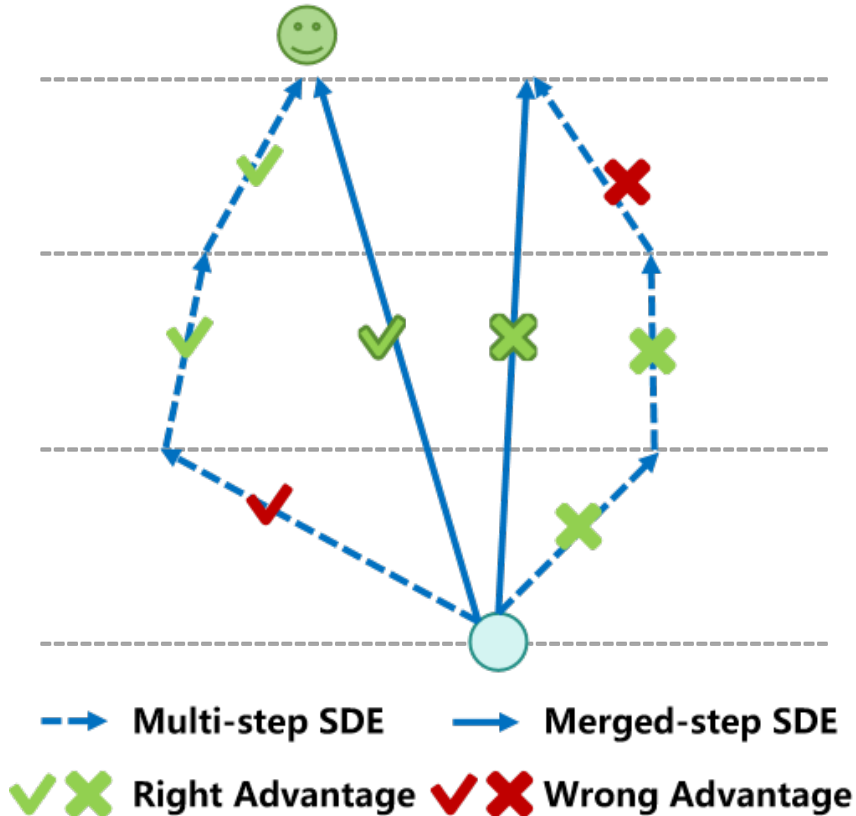


图 3: 连续多步 SDE sampling 会让同一个 final advantage 同时对应多个 timesteps, 从而产生 ambiguous reward signal; E-GRPO 通过 merged-step SDE sampling 把最终 advantage 对齐到一个 merged SDE step, 降低单步 credit assignment 的歧义。图源: [E-GRPO arXiv HTML Figure 3](#)。

这里和 DenseGRPO 不一样: DenseGRPO 是想给每个 step 估计更密的 reward gain; E-GRPO 是先把低信息量 steps 合成一个有意义的 exploration event, 再在这个 event 内做 group-relative advantage。它不是“更细”, 而是“该细的地方细, 该合的地方合”。

1.6 5. 算法流程: 训练时到底怎么做

可以把 E-GRPO 的训练流程压缩成五步:

1. 计算所有 timesteps 的 entropy proxy $h(t_k)$ 。
2. 根据阈值 τ 识别 high-entropy steps 和 low-entropy steps。
3. 对连续 low-entropy steps 自适应选择 l_n , 形成 consolidated SDE steps。
4. 对非 active steps 使用 ODE sampling, 对 consolidated active steps 使用 SDE sampling。
5. 在每个 merged-step group 内计算 advantage 和 clipped GRPO objective。

因此, E-GRPO 实际上把 rollout policy 改成了一个 ODE/SDE hybrid:

ODE on uninformative steps + SDE on entropy-calibrated merged steps.

直觉: 它不是减少采样步数本身, 而是减少“无效随机性”和“无效 RL 更新”。ODE steps 负责稳定推进生成, SDE merged steps 负责真正探索。

1.7 6. 实验设置

论文的实验设置如下:

- **Backbone:** FLUX.1-dev。
- **数据集:** HPD, 约 103K training prompts 和 400 testing prompts。
- **采样步数:** 总采样步数 $T = 16$ 。
- **噪声参数:** $a = 0.7$ 。
- **默认 entropy threshold:** $\tau = 2.2$ 。
- **训练步数:** 300 iterations。

- **训练 reward:** 先使用 HPS-v2.1; 为了缓解 reward hacking, 又使用 HPS-v2.1 + CLIP 的联合 reward。
- **评测指标:** HPS、CLIP、PickScore、ImageReward。

这套设置有一个值得注意的点: 作者明确指出只用 HPS 训练可能导致 reward hacking, 所以主结果里同时报告了单 reward 和 HPS+CLIP 两种训练方式。

1.8 7. 主要结果: E-GRPO 的收益在哪里

1.8.1 7.1 单 HPS reward 下, HPS 指标最好

在只用 HPS-v2.1 作为训练 reward 时, E-GRPO 在 HPS 上达到 **0.391**, 高于几个主要 baseline:

方法	HPS	CLIP	PickScore	ImageReward
FLUX.1-dev	0.311	0.388	0.231	1.089
DanceGRPO	0.353	0.375	0.228	1.233
MixGRPO	0.378	0.358	0.225	1.266
GranularGRPO	0.385	0.355	0.229	1.313
TempFlowGRPO	0.382	0.357	0.231	1.264
E-GRPO	0.391	0.355	0.232	1.324

论文强调, E-GRPO 相比 DanceGRPO 在 HPS 上提升约 **10.8%**。但我更关注另一个现象: HPS 提高的同时, CLIP 并没有同步提高, 说明单 reward 训练确实存在向 HPS scorer 偏移的风险。

1.8.2 7.2 HPS+CLIP 联合 reward 下, 泛化指标更稳

在 HPS+CLIP 联合训练下, E-GRPO 的表现更有说服力:

方法	HPS	CLIP	PickScore	ImageReward
FLUX.1-dev	0.311	0.388	0.231	1.089
DanceGRPO	0.331	0.389	0.227	1.128
MixGRPO	0.363	0.399	0.230	1.436
GranularGRPO	0.377	0.400	0.236	1.490
BranchGRPO	0.342	0.384	0.230	1.243
TempFlowGRPO	0.310	0.388	0.230	1.106
E-GRPO	0.382	0.401	0.237	1.494

论文报告, 相比 DanceGRPO, E-GRPO 在 ImageReward 上提升 **32.4%**, 在 PickScore 上提升 **4.4%**。这说明 entropy-guided step selection 不只是刷某一个训练 reward, 而是能带来更广泛的人类偏好指标改善。

1.8.3 7.3 训练曲线: 早期增长更快, 收敛更平滑

论文还展示了 reward trajectory: E-GRPO 早期 reward 增长更快, 最终 reward 更高, 曲线也更平滑。

这和方法假设是一致的: 如果训练集中在 high-entropy / consolidated informative steps 上, 那么每次更新都更可能改变真正影响 reward 的生成方向, 而不是在低熵后期步骤上浪费梯度。

1.8.4 7.4 可视化结果: 语义和细节更均衡

论文的可视化对比显示, E-GRPO 相比 baseline 更容易同时保留 prompt 语义和细粒度视觉细节。这个现象和前面的机制解释是一致的: high-entropy steps 负责更大的语义探索, merged-step SDE 又避免后期低熵步骤把 reward 噪声放大成错误更新。

1.9 8. 消融: 哪些设计真的重要

1.9.1 8.1 Adaptive merging 优于固定步数 merging

论文比较了固定 2-step、4-step、6-step merging 和 adaptive merging:

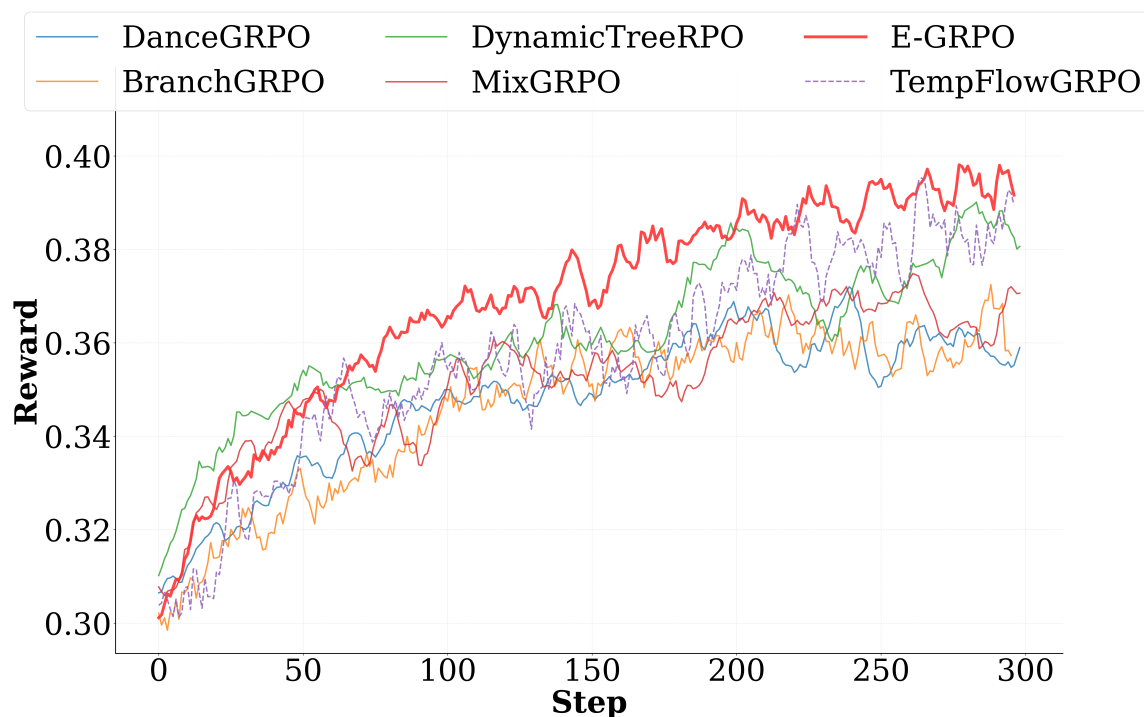


图 4: E-GRPO 与 baseline 的训练 reward 曲线对比。曲线说明 entropy-guided exploration 能更快提升训练 reward, 并且相比全 timestep 更新更稳定。图源: [E-GRPO arXiv HTML Figure 4](#)。

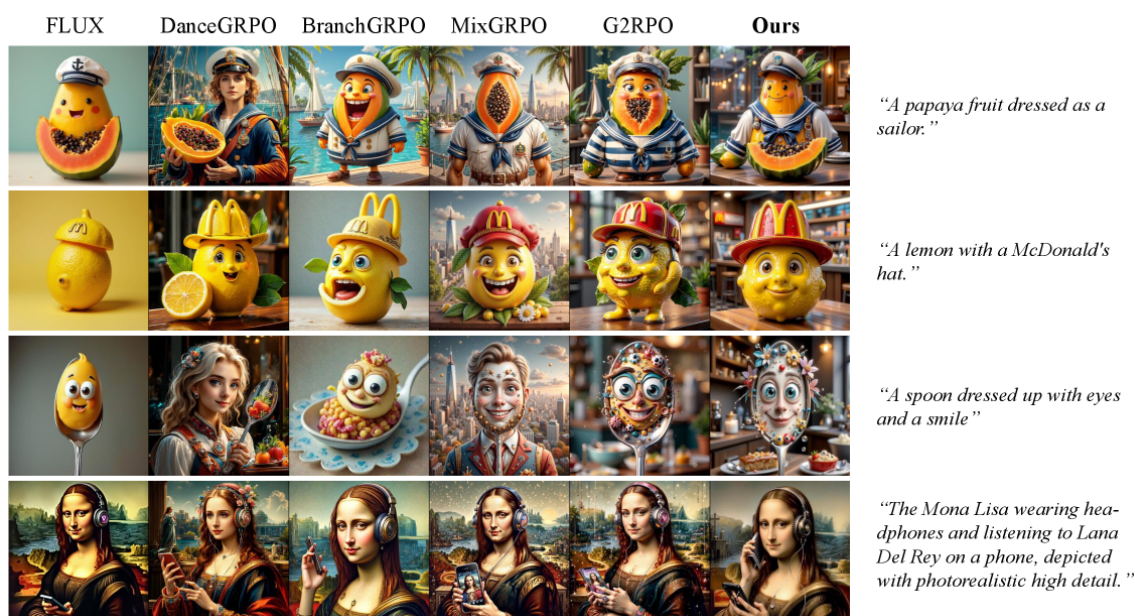


图 5: E-GRPO 与其他 baseline 的生成结果可视化对比。论文认为 E-GRPO 能更好地整合语义对齐和细节质量。图源: [E-GRPO arXiv HTML Figure 5](#)。

Merging strategy	HPS	CLIP	PickScore	ImageReward
2-step	0.382	0.290	0.232	1.223
4-step	0.374	0.302	0.230	1.216
6-step	0.337	0.372	0.226	1.298
Adaptive	0.391	0.355	0.232	1.324

这说明固定 merge 的问题在于不看局部 entropy: 合并太少, 探索仍然不足; 合并太多, 单个 SDE event 过于随机, reward 方向又会变模糊。Adaptive merging 的优势就是把每个 active step 的探索强度拉回到一个合适范围。

1.9.2 8.2 只训前 8 个 high-entropy steps 最好

论文还比较了训练不同 timestep subset:

训练 steps	HPS	CLIP	PickScore	ImageReward
First 4 Steps	0.370	0.348	0.231	1.252
First 8 Steps	0.391	0.355	0.232	1.324
Second 8 Steps	0.357	0.381	0.231	1.250
Full Steps	0.366	0.359	0.231	1.169

结论非常清楚: 前 8 个 high-entropy steps 已经足以获得最强性能; 训练所有 steps 不但成本更高, 效果还不如只训前半段; 只训后 8 个 low-entropy steps 则明显更弱。

我会这样记这组消融: E-GRPO 不是“少训一些 steps 也能省钱”这么简单, 而是证明了 **后半段低熵 steps 的 RL 信号质量确实差**。全步骤训练把有用信号和低质量信号混在一起, 反而拖累优化。

1.10 9. 和 OP-GRPO / DenseGRPO / TempFlow-GRPO 的关系

把 E-GRPO 放进这条 Flow-GRPO 后训练主线里, 它的位置大概是:

论文	主要改哪里	核心问题
TempFlow-GRPO	temporal weighting / branching	哪个 timestep 更该分到 credit
DenseGRPO	dense reward gain	terminal reward 怎样分给每个 step
OP-GRPO	off-policy replay	已经采出来的高质量 trajectory 能否复用
E-GRPO	entropy-aware exploration	哪些 steps 值得随机探索和 RL 更新

- **和 OP-GRPO:** 两者都关心效率, 但 OP-GRPO 是跨 iteration 的样本复用, E-GRPO 是单条 trajectory 内的 step selection。前者解决“好轨迹别浪费”, 后者解决“低熵步骤别乱训”。
- **和 DenseGRPO:** DenseGRPO 把 reward 变密, E-GRPO 把探索步骤变少但更有信息量。一个偏 reward attribution, 一个偏 exploration attribution。
- **和 TempFlow-GRPO:** TempFlow-GRPO 也注意到 timestep 不应等权; E-GRPO 则从 SDE entropy 出发, 给出更直接的 step merging 和 ODE/SDE hybrid sampling 机制。

如果组合使用: 一个自然方向是 OP-GRPO 复用高 reward trajectories, 但只复用 / 更新其中 E-GRPO 认为 high-entropy 或 consolidated informative 的 steps。这样可能同时减少采样成本和无效更新成本。

1.11 10. 我认为它的局限

E-GRPO 的思想很清楚, 但也有几个边界:

1. **Entropy proxy 不等于真实任务重要性。** 高熵 step 更可能产生差异, 但差异大不一定等于对某个 reward 最关键; 任务相关 credit 仍然需要 reward 侧方法补充。
2. **阈值 τ 仍是超参数。** 论文默认设为 2.2, 并在附录做了阈值消融, 但换模型、换 scheduler、换采样步数后可能需要重新校准。
3. **主要验证在 FLUX.1-dev + HPD。** 它是否同样适用于视频、图像编辑、3D 或 unified multimodal generation, 还需要更多证据。
4. **reward hacking 没有被根治。** 论文自己也承认 reward design 仍是瓶颈; E-GRPO 能减少无效步骤优化, 但不能保证 reward 本身完全符合人类偏好。

1.12 11. 读完后的记忆钩子

我最后会这样概括: OP-GRPO 说“昂贵采样出来的好轨迹别只用一次”; E-GRPO 说“昂贵轨迹里的每一步也别都当成有效探索”。前者是 trajectory-level efficiency,

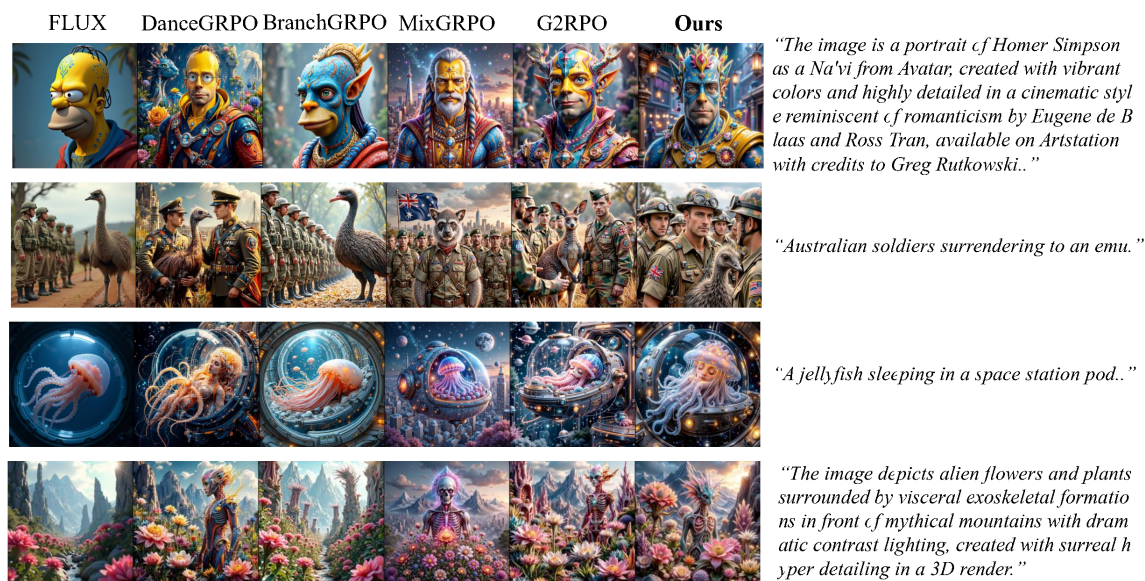


图 6: E-GRPO 的 failure cases。这个图提醒我们: entropy-aware step selection 能提升 RL 更新效率, 但不能替代更可靠的 reward model、prompt grounding 和视觉一致性约束。图源: [E-GRPO arXiv HTML Figure 8](#)。

后者是 timestep-level exploration efficiency。两篇连起来, 正好构成 Flow-GRPO 采样效率支线的两个层级。

1.13 参考来源

- 论文主页: [E-GRPO: High Entropy Steps Drive Effective Reinforcement Learning for Flow Models](#)。
- arXiv HTML: [arXiv HTML version](#)。
- 论文配图: [arXiv HTML figures](#), 已用于本文的 entropy analysis、pipeline、reward ambiguity、training curve、visual comparison 和 failure cases 图示。
- 代码仓库: [shengjun-zhang/VisualGRPO](#)。